

^{"z"}
ZED Attack Proxy
↑
(ZAP)
↑



Kameswari Chebrolu
Department of CSE, IIT
Bombay



<https://ih1.redbubble.net/image.124708997.7614/st,sma11,507x507-pad,600x600,f8f8f8.jpg>

Types of Security Testing

Generic Security Testing (not just Web Applications)

- Vulnerability Assessment
- Penetration Testing
- Runtime Testing
- Code Review

- Vulnerability Assessment: Involves scanning and analyzing a system for known vulnerabilities
 - System: applications, servers, or networks
 - Automated using tools like Nessus, OpenVas etc
 - Focuses on known vulnerabilities (e.g., outdated software, misconfigurations, unpatched systems)
 - A report is generated detailing findings and remediation suggestions
 - Vulnerabilities are also categorized based on severity (low, medium, high)
 - **Does not confirm if vulnerabilities can be exploited**

- Penetration Testing: A simulated attack on the system to identify exploitable vulnerabilities
 - Evaluates system's ability to withstand actual attacks and identify potential security gaps
 - Performed by ethical hackers or penetration testers
 - Can be
 - Black Box (no prior knowledge of the system)
 - White Box (full access to system details) or
 - Gray Box (partial knowledge of the system)
 - Assess the impact of a successful exploit (e.g., data exfiltration, privilege escalation) → root
 - Provides actionable insights and helps prioritize fixes



- Runtime Testing: Test system during its execution (runtime) from an end-user perspective
 - Simulates real-world usage scenarios to identify security flaws
 - Application or system run in a controlled environment
 - Simulated or real users interact with the system
 - Monitor behaviors such as insecure data handling, improper session management, or authentication bypasses etc
 - Identifies runtime-specific vulnerabilities, such as logic flaws, unprotected endpoints, or exposed sensitive data
 - Includes DAST: Dynamic Application Security Testing (black box testing)

- Code Review: A detailed examination of source code to detect security vulnerabilities and secure coding practices
 - Helps find and fix vulnerabilities in code before they can be exploited
 - Performed manually or via Static Application Security Testing (SAST) tools like SonarQube, Checkmarx, or Veracode
 - Document findings with recommendations for remediation
 - Developers should address the identified issues and re-submit the code for verification

Aspect	Vulnerability Assessment	Penetration Testing	Runtime Testing	Code Review
Focus	Known vulnerabilities	Exploitable vulnerabilities	Runtime-specific flaws	Insecure coding practices
Approach	Automated scans	Simulated attacks	End-user interaction	Manual/automated code analysis
Outcome	List of weaknesses	Validation of exploitable gaps	Identification of runtime issues	Secure code
Tools	Nessus, Qualys, OpenVAS	Metasploit, Burp Suite, ZAP	DAST tools like OWASP ZAP	SAST tools like SonarQube
Expertise Required	Low to moderate	High (ethical hackers)	Moderate	High (developers/security experts)
Timing	Post-deployment	Post-deployment	During runtime	Pre-deployment

OWASP ZAP

- An open-source web application security pen-testing tool
 - Developed by the Open Web Application Security Project (OWASP)
 - Widely used for detecting vulnerabilities in web applications and APIs
 - Runs on all major platforms, including Windows, macOS, and Linux

Key Features

- Automated Scanning:
 - Passive Scan: Observes traffic without interacting directly with the application
 - Active Scan: Actively probes the application by sending crafted payloads to discover vulnerabilities
- Fuzzing: Performs fuzz testing by sending random or malformed data to find input handling vulnerabilities
- Authentication Testing: Supports authentication mechanisms like Password based, OAuth, SAML, cookie-based authentication etc

- API Security Testing: Supports REST, SOAP, and GraphQL API security testing
- Session Management: Tracks and manipulates session data for testing session management vulnerabilities
- Spidering and Crawling: Automatically maps application structure by crawling web pages or executing JavaScript (via the AJAX spider)

- Custom Scripts: Users can extend its functionality using scripts written in JavaScript, Python, or Groovy
- Reporting: Generates detailed reports, including the severity of detected vulnerabilities and recommendations for fixes

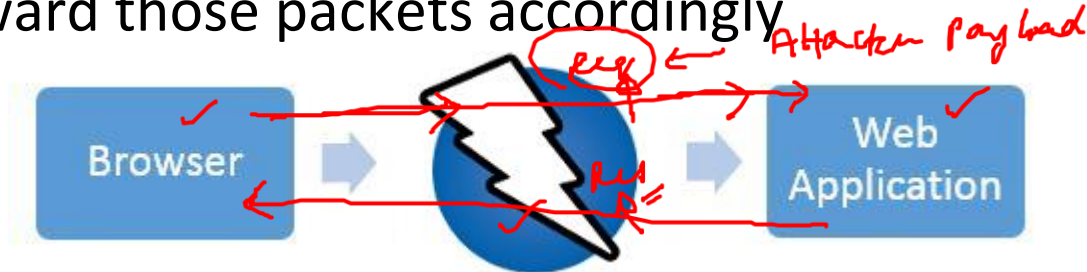
Common Web Vulnerabilities

ZAP can detect many web vulnerabilities

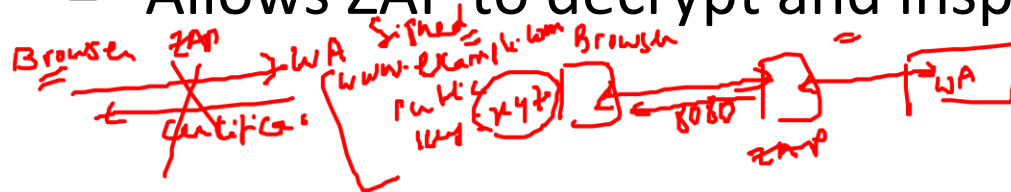
- Cross-Site Scripting (XSS)
- SQL Injection
- Broken Authentication
- Security Misconfigurations
- Sensitive Data Exposure
- Cross-Site Request Forgery (CSRF)
- Directory Traversal etc

ZAP Architecture

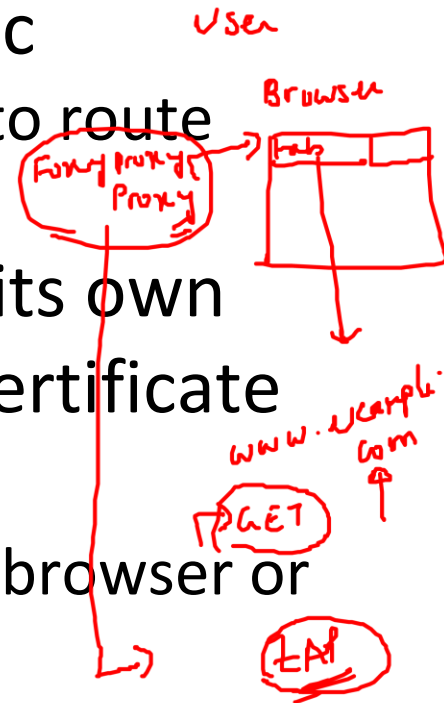
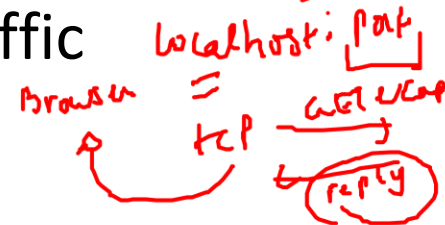
- ZAP is a “manipulator-in-the-middle proxy”
 - Stands between tester’s browser and web application
 - Intercepts and inspects messages sent between browser and web application (bi-directional)
 - Will also modify the contents if needed and then forward those packets accordingly

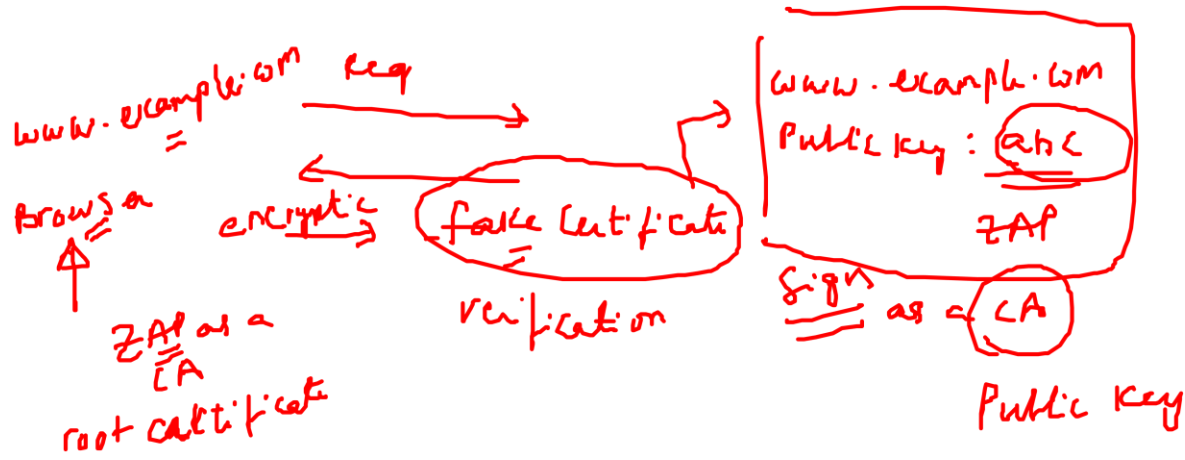
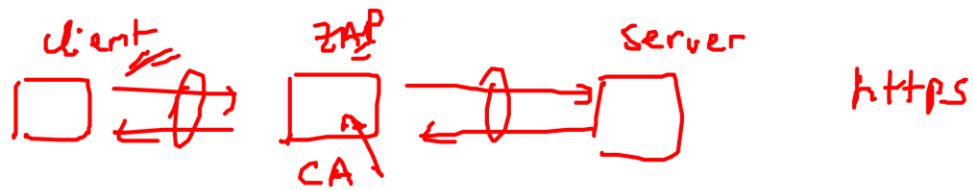


- ZAP acts as a proxy server that listens on a specific port (default: 8080) for HTTP/HTTPS traffic
 - Both the client and ZAP must be configured to route traffic through this proxy
- Certificate Authority (CA): ZAP generates its own SSL/TLS certificate and acts as a trusted Certificate Authority (CA)
 - Client must install ZAP's CA certificate in the browser or system to avoid SSL/TLS errors
 - Allows ZAP to decrypt and inspect HTTPS traffic

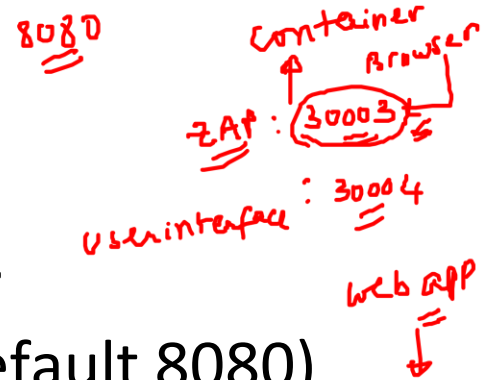


http
https





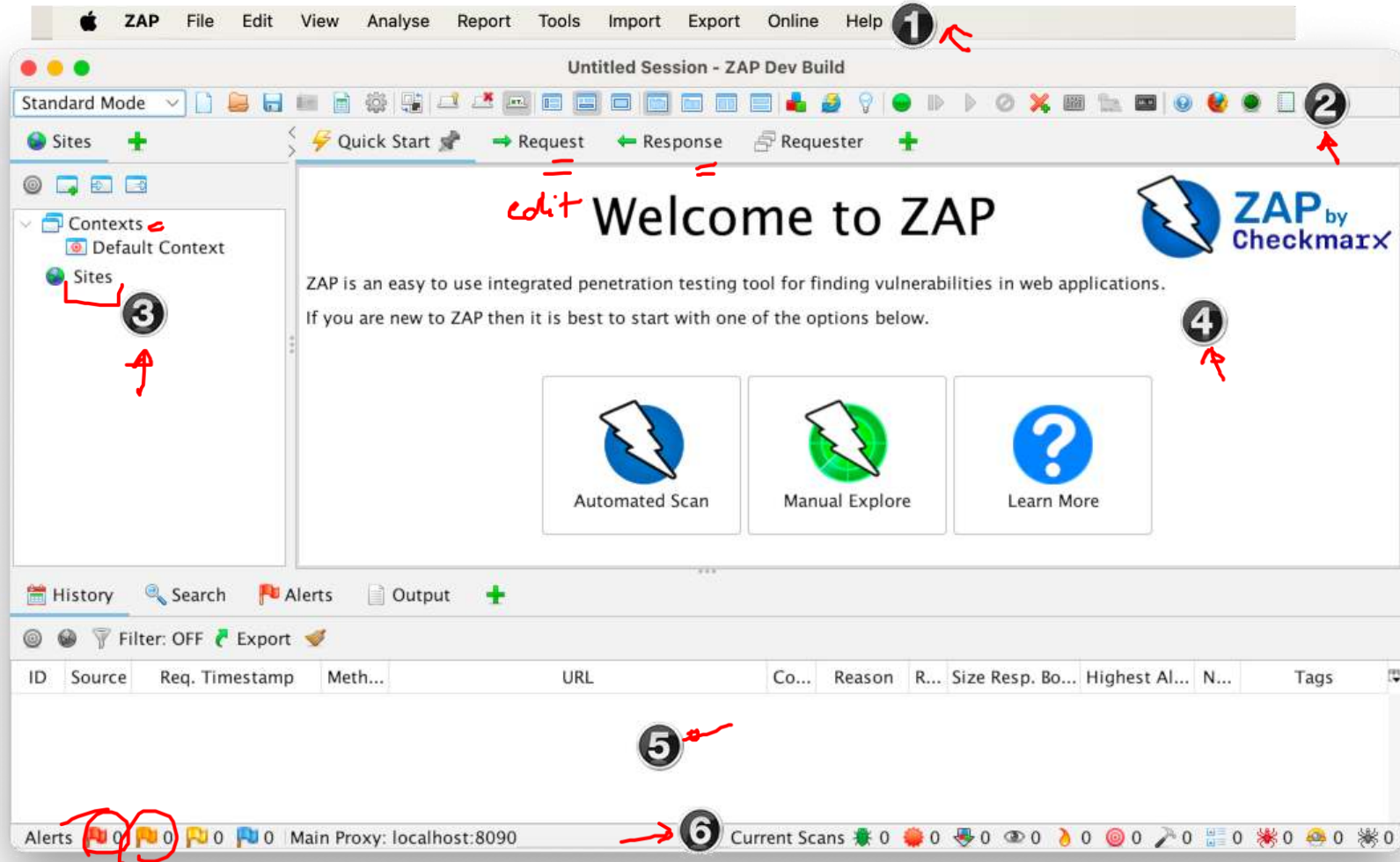
cLab App (Labs)



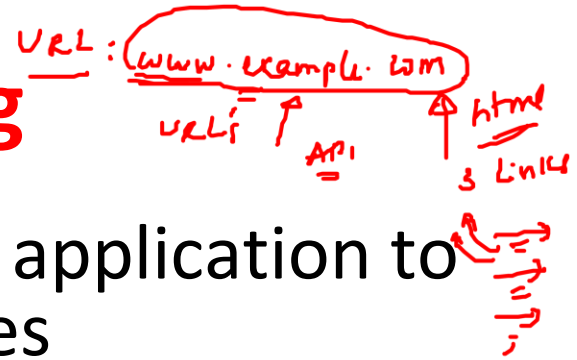
- ZAP is running locally in the container
 - Listens on port 30003 (different from default 8080) for incoming traffic from browser
 - Web interface is on a different port 30004
 - “<http://localhost:30004/zap/>” ✓
 - Foxyproxy plugin for browser needs to be configured to send/receive traffic to/from port 30003
- Vulnerable website is also running locally and is on port 30000

Desktop UI

1. Menu Bar – Provides access to many of the automated and manual tools
2. Toolbar – Includes buttons which provide easy access to most commonly used features
3. Tree Window – Displays the Sites tree and the Scripts tree
4. Workspace Window – Displays requests, responses, and scripts and allows you to edit them
5. Information Window – Displays details of the automated and manual tools
6. Footer – Displays a summary of the alerts found and the status of the main automated tools



Spidering/Crawling



- Spidering refers to crawling the web application to discover its structure, links, and pages
- ZAP has two spidering methods:
- Traditional Spidering (HTML Spider): Parses application's HTML to find links, forms, and other navigational elements
 - Follows these links recursively to map the application
 - Cannot handle dynamically loaded content (e.g., JavaScript-heavy websites)
 - Works for applications with minimal JavaScript or no reliance on client-side frameworks

- AJAX Spidering: Uses a headless browser (e.g. Selenium) to execute JavaScript and interact with dynamically loaded content
 - Can discover pages and links rendered only through client-side JavaScript
 - Suitable for modern web applications built with frameworks like React, Angular, or Vue.js
 - Can handle complex, dynamic interactions such as dropdowns, modals, and client-side routing
 - However, slower than traditional spidering

Manual Exploration

- Helps complement automated crawling (spidering) when testing a web application's security
- Testers will interact with the application directly
 - Navigate through its pages, and trigger functionality that automated tools might miss

- Below may not be fully navigated by automated crawlers
 - Complex workflows, like multi-step forms or authentication-based pages or interacting with specific user roles
 - AJAX requests or client-side frameworks (e.g., React, Angular)
 - Can create hidden or conditional elements
 - Functionality that's not accessible via links (e.g., API endpoints, user interactions like drag-and-drop)

- Perform tasks such as below when exploring the application
 - Logging in with different user roles
 - Submitting forms and entering varying input
 - Triggering dynamic or conditional UI elements
 - Entering invalid data
- ZAP can capture all this traffic for scanning

→ web pages

Automated Scanning

→ 2nd
→ 1st

- Passive Scanning: analyzes HTTP traffic (requests and responses) as it passes without explicitly interacting with the target application
 - Does not actively probe or send malicious payloads.
 - Looks for vulnerabilities such as missing security headers, information leaks, insecure cookies, or improper server configurations
 - Non-intrusive and safe for production environments
 - Helps assess baseline security issues

- Active Scanning: Actively probes application for vulnerabilities by sending crafted requests and analyzing responses  (attacker payload)
 - Uses known attack techniques (e.g., SQL injection, XSS, directory traversal) to test the application's defenses
 - Intrusive and may disrupt the application
 - More thorough than passive scanning but should typically be used in testing environments
 - Helps perform penetration testing on staging or development environments
 - "Pen testing is more thorough and often involves manual testing of more sophisticated attacks also"

References

<https://www.zaproxy.org/getting-started/>